ARTI 308-406: MACHINE LEARNING

Dr. Ito Wasito

Course Orientation
Linear Algebra Bridge to Machine Learning

Contents selectively adapted from textbook: Probabilistic Machine Learning: An Introduction by Kevin Murphy, 2022 and Mathematics for Machine Learning by Deisenroth et al, 2020.

1/24/2026

Topics

1. Overview on Course Learning Outcomes, Assessment, Tools and Expectations
2. Linear Algebra as Foundation for Machine Learning
3. Linear Maps and Matrix operations
4. Data Representation in Machine Learning
5. Motivation
6. Mini Activity

Course Overview: Course Learning Outcomes (CLO)

At the end of this course, students should be able to:

Knowledge and Understanding

1. Identify examples of classification and regression tasks, including the available input features and output to be predicted.
2. Describe the algorithmic basis of machine learning algorithms and concepts

Skills :

1. Explain implementation issues in learning algorithms and the evaluation of their performances
2. Apply machine learning techniques and algorithms to real life problems
3. Compare and contrast machine learning concepts and techniques
4. Communicate effectively through scientific and technical documentation or presentation describing project activities and outcomes.

Values:

1. Demonstrate abilities to participate effectively in team activities.

Course Overview: Assessment

Your course grade will be calculated as follows:

Item	Weight (%)
Quizzes	10%
Lab Assignments	15%
Group project	15%
Mid Term exam	20%
Final exam	40%
Total	100%

Course Overview: Tools and Expectations

Tools:

Programming Language: Python
Code Editor: Visual Studio Code or Jupyter Notebook
Developer Platform: Github

Expectations:

1. Students can demonstrate ability to apply their machine learning knowledge and skills to solve lab assignments using Python code programming with powerful code editor on the Github developer platform.
2. Students can implement their knowledge, skills and values from this course to solve any real problems those require Machine Learning techniques as problem solving.

Linear Algebra as a Foundation for Machine Learning

Linear Algebra:

When formalizing intuitive concepts, a common approach is to construct a set of objects (symbols) and a set of rules to manipulate these objects. This is known as an ***algebra (Reference Book 1, Deisonrorth et al, 2020).***

Linear algebra is the study of matrices and vectors (**Main textbook, Kevin Murphy, 2022**).

There are three major components of a machine learning system: **data, models, and learning.**

Linear Algebra has mainly important role in **data** component in Machine Learning system.

For **models and learning parts**, Machine Learning system heavily depend on Statistics, Probability, Information Theory, Decision Theory and Optimization

Linear Algebra Data Representation

Vectors

A vector $x \in R^n$ is a list of n numbers, usually written as a column vector

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

The vector of all ones is denoted 1. The vector of all zeros is denoted 0.

The unit vector e_i is a vector of all 0's, except entry i, which has value 1:

$$e_i = (0, \ldots, 0, 1, 0, \ldots, 0)$$

This is also called a one-hot vector

Linear Algebra Data Representation

Matrices

A matrix A ∈ R^{mxn} with *m* rows and *n* columns is a *2d* array of numbers, arranged as follows:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

If m = n, the matrix is said to be **square**

We use the notation A_{ij} or $A_{i,j}$ to denote the entry of A in the ith row and j^{th} column.

The transpose of a matrix results from "flipping" the rows and columns.

Given a matrix A ∈ R^{mxn}, its transpose, written $A^T \in R^{nxm}$ is defined as $(A^T)_{ij} = A_{ji}$

The following properties of transposes are easily verified:

$$(\mathbf{A}^\mathsf{T})^\mathsf{T} = \mathbf{A}$$
$$(\mathbf{AB})^\mathsf{T} = \mathbf{B}^\mathsf{T}\mathbf{A}^\mathsf{T}$$
$$(\mathbf{A}+\mathbf{B})^\mathsf{T} = \mathbf{A}^\mathsf{T} + \mathbf{B}^\mathsf{T}$$

If a square matrix satisfies $A = A^T$, it is called ***symmetric***.

Linear Algebra Data Representation

Tensors

In Machine Learning terminology, a ***tensor*** is just a generalization of a 2d array to more than 2 dimensions, as illustrated in Figure 1.

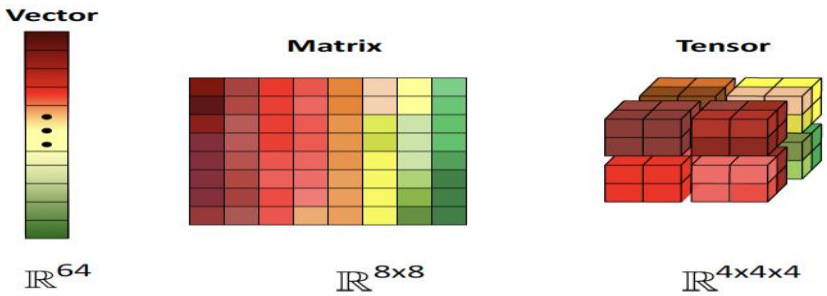


Figure 1. Illustration of a 1d vector, 2d matrix, and 3d tensor. The colors are used to represent individual entries of the vector; this list of numbers can also be stored in a 2d matrix, as shown. (In this example, the matrix is layed out in column-major order, which is the opposite of that used by Python.) We can also reshape the vector into a 3d tensor, as shown in Figure 2 in the next slide

We can reshape a matrix into a vector by stacking its columns on top of each other, as shown in Figure 1. This is denoted by

$$\mathrm{vec}(\mathbf{A}) = [\mathbf{A}_{:,1}; \cdots ; \mathbf{A}_{:,n}] \in \mathbb{R}^{mn\times1}$$

Order of Tensor

- The number of dimensions is known as the order or rank of the tensor.
- In mathematics, tensors can be viewed as a way to define multilinear maps, just as matrices can be used to define linear functions, although we will not need to use this interpretation.

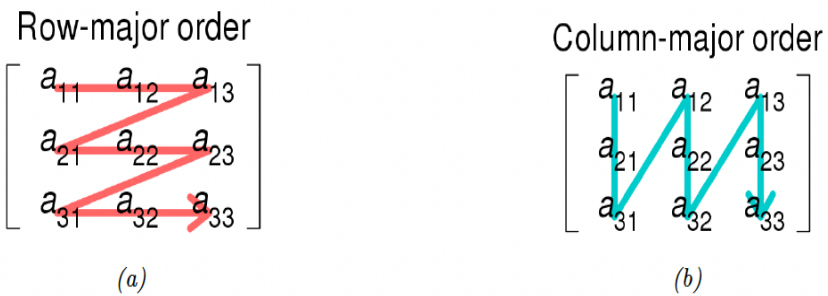


Figure 2. Illustration of (a) row-major vs (b) column-major order.

Reshape a vector into a matrix

There are two choices for how to do this, known as row-major order (used by languages such as Python and C++) and column-major order (used by languages such as Julia, Matlab, R and Fortran). See Figure 2 for an illustration of the difference.

Review on Matrix Multiplication

The product of two matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$ is the matrix

$$\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p},$$

where

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

- Note that in order for the matrix product to exist, the number of columns in A must equal the. number of rows in B.
- Matrix multiplication generally takes O(mnp) time, although faster methods exist.
- In addition, specialized hardware, such as GPUs and TPUs, can be leveraged to speed up matrix multiplication significantly, by performing operations across the rows (or columns) in parallel.

Norm of Vector and Matrices

- Vector norms

A **norm** of a vector $\|x\|$ is, informally, a measure of the "length" of the vector. More formally, a norm is any function $f : \mathbb{R}^n \to \mathbb{R}$ that satisfies 4 properties:

- For all $x \in \mathbb{R}^n$, $f(x) \geq 0$ (non-negativity).
- $f(x) = 0$ if and only if $x = 0$ (definiteness).
- For all $x \in \mathbb{R}^n$, $t \in \mathbb{R}$, $f(tx) = |t|f(x)$ (absolute value homogeneity).
- For all $x, y \in \mathbb{R}^n$, $f(x + y) \leq f(x) + f(y)$ (triangle inequality).

Consider the following common examples:

p-norm $\|x\|_p = \left(\sum_{i=1}^n |x_i|^p\right)^{1/p}$, for $p \geq 1$.

2-norm $\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$, also called Euclidean norm. Note that $\|x\|_2^2 = x^\mathsf{T} x$.

1-norm $\|x\|_1 = \sum_{i=1}^n |x_i|$.

Max-norm $\|x\|_\infty = \max_i |x_i|$.

Norm of Vector and Matrices

- Matrix norms

Suppose we think of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ as defining a linear function $f(x) = \mathbf{A}x$. We define the **induced norm** of $\mathbf{A}$ as the maximum amount by which f can lengthen any unit-norm input:

$$||\mathbf{A}||_p = \max_{\boldsymbol{x} \neq \mathbf{0}} \frac{||\mathbf{A}\boldsymbol{x}||_p}{||\boldsymbol{x}||_p} = \max_{||\boldsymbol{x}||=1} ||\mathbf{A}\boldsymbol{x}||_p$$

Typically $p = 2$, in which case

$$||\mathbf{A}||_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\mathsf{T}\mathbf{A})} = \max_i \sigma_i$$

where $\lambda_{\max}(\mathbf{M})$ is the largest eigenvalue of $\mathbf{M}$, and σ_i is the i'th singular value.

Dot Product and Distance Matrix

Given two vectors $x, y \in \mathbb{R}^n$, the quantity $x^\mathsf{T} y$, called the **inner product**, **dot product** or **scalar product** of the vectors, is a real number given by

$$\langle x, y \rangle \triangleq x^\mathsf{T} y = \sum_{i=1}^{n} x_i y_i.$$

Note that it is always the case that $x^\mathsf{T} y = y^\mathsf{T} x$.

Distance matrix

Let $\mathbf{X}$ be $N_x \times D$ datamatrix, and $\mathbf{Y}$ be another $N_y \times D$ datamatrix. We can compute the squared pairwise distances between these using

$$\mathrm{D}_{ij} = (x_i - y_j)^\mathsf{T}(x_i - y_j) = \|x_i\|^2 - 2x_i^\mathsf{T} y_j + \|y_j\|^2$$

Let us now write this in matrix form. Let $\hat{x} = [\|x_1\|^2; \cdots; \|x_{N_x}\|^2] = \mathrm{diag}(\mathbf{X}\mathbf{X}^\mathsf{T})$ be a vector where each element is the squared norm of the examples in $\mathbf{X}$, and define $\hat{y}$ similarly. Then we have

$$\mathbf{D} = \hat{x}\mathbf{1}_{N_y}^\mathsf{T} - 2\mathbf{X}\mathbf{Y}^\mathsf{T} + \mathbf{1}_{N_x}\hat{y}^\mathsf{T}$$

In the case that $\mathbf{X} = \mathbf{Y}$, we have

$$\mathbf{D} = \hat{x}\mathbf{1}_{N}^\mathsf{T} - 2\mathbf{X}\mathbf{X}^\mathsf{T} + \mathbf{1}_{N}\hat{x}^\mathsf{T}$$

This vectorized computation is often much faster than using for loops.

Linear Maps

Linear independence, spans and basis sets

A set of vectors $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots \boldsymbol{x}_n\}$ is said to be (linearly) independent if no vector can be represented as a linear combination of the remaining vectors. Conversely, a vector which can be represented as a linear combination of the remaining vectors is said to be (linearly) dependent

For example, if $x_n = \sum_{i=1}^{n-1} \alpha_i x_i$ for some $\{\alpha_1, \ldots, \alpha_{n-1}\}$ Then $\boldsymbol{x}_n$ is dependent on $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{n-1}\}$ otherwise, it is independent of $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{n-1}\}$.

The span of a set of vectors $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots \boldsymbol{x}_n\}$ is the set of all vectors that can be expressed as a. linear combination of $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$

A basis $\mathcal{B}$ is a set of linearly independent vectors that spans the whole space, meaning that

$$\text{span}(\mathcal{B}) = \mathbb{R}^n$$

Linear Maps

- A linear maps or linear transformation is any function

$$f : \mathcal{V} \to \mathcal{W}$$

such that $f(\boldsymbol{v}+\boldsymbol{w}) = f(\boldsymbol{v})+f(\boldsymbol{w})$ and $f(a\ \boldsymbol{v}) = a\ f(\boldsymbol{v})$ for all $\boldsymbol{v}, \boldsymbol{w} \in \mathcal{V}$.

Once the basis of $\mathcal{V}$ is chosen, a linear map $f : \mathcal{V} \to \mathcal{W}$

is completely determined by specifying the **images** of the **basis vectors**, because any element of $\mathcal{V}$ can be expressed uniquely as a **linear combination** of them.

Datasets, Labels, Parameters and Bias

Suppose $\mathcal{V} = \mathbb{R}^n$ and $\mathcal{W} = \mathbb{R}^m$, we can compute $f(v_i) \in \mathbb{R}^m$ for each basis vector in $\mathcal{V}$, and store these along the columns of an $m \times n$ matrix A.

For example, we can then compute $y = f(x) \in \mathbb{R}^m$ for any $x \in \mathbb{R}^n$ as follows:

$$y = \left(\sum_{j=1}^{n} a_{1j} x_j, \ldots, \sum_{j=1}^{n} a_{mj} x_j \right)$$

This corresponds to multiplying the vector x by the matrix A: $y = \mathrm{A}x$

In **Machine Learning**: Matrix A, Vector y and vector x represent ***datasets, labels and parameters,*** respectively. Sometimes, when we use Machine Learning as Prediction model as follows

$$\mathbf{y} = \mathbf{A}\boldsymbol{x} + \boldsymbol{b} \qquad \textbf{(1)}$$

The ***vector b*** in the Machine Learning we called as ***bias***. We will explore more details about it the next section.

For this exploration, we can see that basically datasets in Machine Learning systems heavily depend on Linear Algebra Theory!

This shows the beauty of Mathematics in real world problems.

Instance and Feature

- We assume that our data can be read by a computer, and represented

adequately in a numerical format as the following example (taken from reference book no 1)

Name	Gender	Degree	Postcode	Age	Annual salary
Aditya	M	MSc	W21BG	36	89563
Bob	M	PhD	EC1A1BA	47	123543
Chloé	F	BEcon	SW1A1BH	26	23989
Daisuke	M	BSc	SE207AT	68	138769
Elisabeth	F	MBA	SE10AA	33	113888

Table2. Tabular data without numerical format

Data is assumed to be tabular, where we think of each row of the table as representing a particular ***instance*** or **example**, and each column to be a particular ***feature.***

Numerical Representation

- Even when we have data in tabular format, there are still choices to be made to obtain a numerical representation.
- For example, in Table 1, the gender column (a categorical variable) may be converted into numbers 0 representing "Male" and 1 representing "Female". Alternatively, the gender could be represented by numbers −1, +1, respectively (as shown in Table 2 below)

Gender ID	Degree	Latitude (in degrees)	Longitude (in degrees)	Age	Annual Salary (in thousands)
-1	2	51.5073	0.1290	36	89.563
-1	3	51.5074	0.1275	47	123.543
+1	1	51.5071	0.1278	26	23.989
-1	1	51.5075	0.1281	68	138.769
+1	2	51.5074	0.1278	33	113.888

Table2. Converting the feature into numerical representation

Furthermore, it is often important to use ***domain knowledge*** when constructing the representation, such as knowing that university degrees progress from bachelor's to master's to PhD or realizing that the postcode provided is not just a string of characters but actually encodes an area in London.

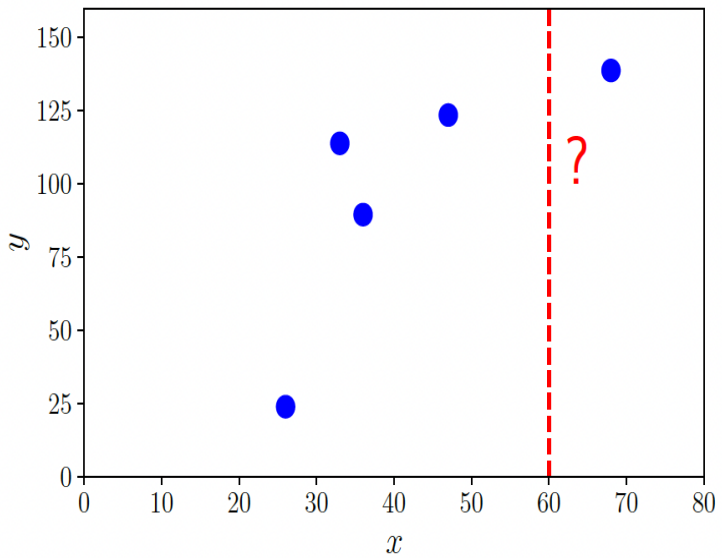
Motivation: Problem to predict labels

- Let us consider the problem of predicting annual salary from age, based on the data in Table 2

In the next lecture, this is called a ***supervised learning*** problem where we have a ***label*** y_n (the salary) associated with each ***example*** $\boldsymbol{x}_n$ (the age).

Representing data as vectors $\boldsymbol{x}_n$ allows us to use concepts from linear algebra.

Figure 3. Data in (x_n, y_n) pairs from the rightmost two columns of Table 2. We are interested in the salary of a person aged sixty (x = 60) illustrated as a vertical dashed red line, which is not part of data.



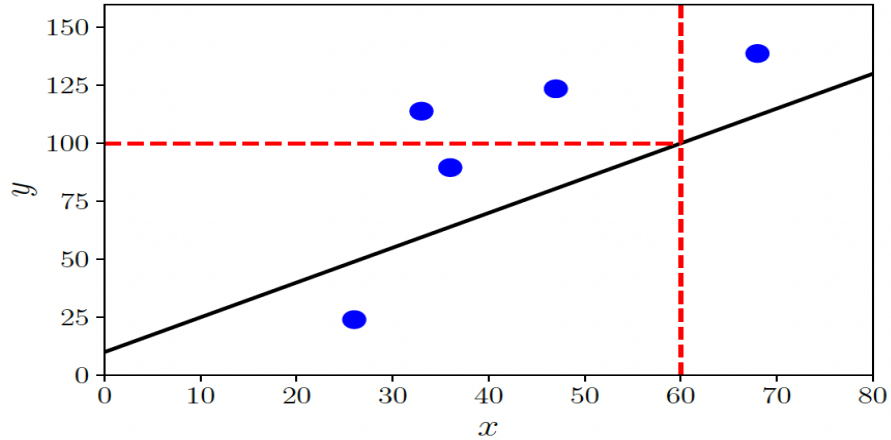
Motivation: Problem to predict labels

- Once we have data in an appropriate vector representation, we can get to the business of constructing a predictive function (known as a ***predictor).***
- This can be written as :

$$f : \mathbb{R}^D \to \mathbb{R}$$

where the input vector x is D-dimensional (has D *features*), and the function f then applied to it (written as $f(x)$) returns a real number.

- Figure 4. Example function (black solid diagonal line) and its prediction at x = 60, i.e., f(60) = 100.



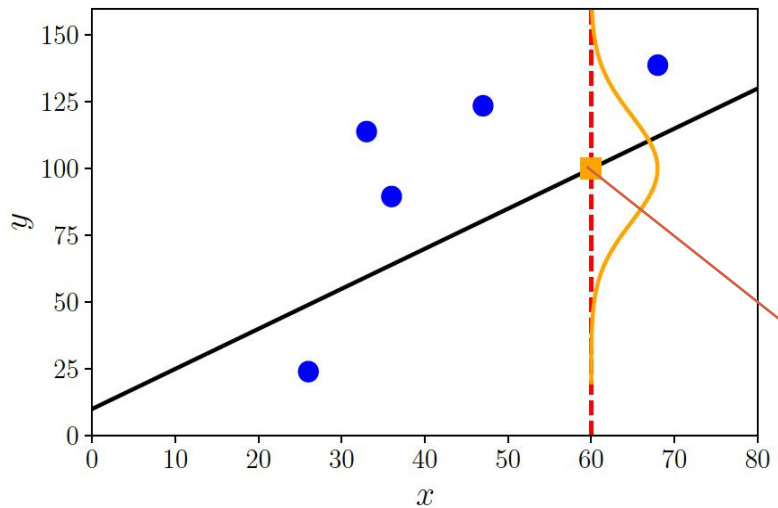
Prediction with uncertainty (bias)

- Consider the special case of linear functions as follows:

for unknown θ and θ_0. $f(\boldsymbol{x}) = \boldsymbol{\theta}^\top \boldsymbol{x} + \theta_0$

Using equation (1) in the slide page 17, *f(x), θ,x and* θ_0 represent labels, parameters, datasets and bias, respectively.

Figure 5. Example function (black solid diagonal line) and its predictive uncertainty at x = 60 (drawn as a Gaussian)



Bias with Gaussian Distribution

Mini Activity: Simple Prediction

1.From the table2, predict Annual Salary values from Age= 50.

2. From Table 2, predict Annual Salary values from Latitude (in degrees)= 51.5072 .

3. From Table 2, predict Annual Salary values from Longitude (in degrees) = 0.1280.

Online Reading

- https://www.ibm.com/think/topics/linear-algebra-for-machine-learning
- https://pabloinsente.github.io/intro-linear-algebra

THANK YOU